CoachAI — Meeting Cheat Sheet

Speak these answers directly. Only what matters.

What is orchestration?

One controller decides which AI agent runs, in what order, and merges their outputs. In my project: Orchestrator manages the session (start / per-turn / end / report), and ConversationManager runs a fixed sequence every turn — Intent Agent, then Knowledge Agent, then Coaching Agent, then Escalation Agent. It's a hard-coded pipeline, not a self-deciding AI.

Are you using an embedding model?

No. Retrieval uses keyword/word-overlap matching, not neural embeddings. It's a lightweight design choice — fast, no extra ML dependency — with embeddings as a planned upgrade once the knowledge base grows.

What is chunking, and how did you do it?

Chunking = splitting documents into smaller pieces so retrieval returns just the relevant part. I split files on blank lines into paragraphs, group them up to 512 characters per chunk, and fall back to sentence-splitting if a paragraph is too long.

What vector database are you using?

None currently. Chunks are held in an in-memory Python list with their tokenized words, matched by keyword overlap. A vector DB (Chroma/FAISS) is the planned next step alongside embeddings.

How does RAG work here end-to-end?

Retrieval: customer message tokenized, compared to every KB chunk by word overlap, top matches picked. Generation: those matched chunks are put into a prompt sent to the LLM, which writes one actionable tip — grounded in real KB content instead of the model guessing.

How does the multi-agent pipeline work?

Per customer message: Intent & Sentiment Agent classifies emotion/intent → Knowledge Agent retrieves + synthesizes a KB tip → Coaching Agent suggests a reply and scores tone → Escalation Agent scores risk. All four combine into one result shown live in the 3-panel console.

Which LLM powers the agents?

Groq API, Llama 3.3 70B as primary, with automatic fallback to smaller models (Llama 3.1 8B, Gemma2 9B, Llama3 8B) if the primary is rate-limited.

Biggest technical trade-off?

Keyword-based retrieval instead of embeddings + a vector database — fast and dependency-light to build, at the cost of missing synonym matches. Clean, well-scoped upgrade for the next milestone.